企业智能知识助理 & 工作流 Agent 系统设计

用 MCP + Agent + RAG + LangChain：完整走一遍 AI 大模型系统设计

一、项目背景 & 目标

假设你要为一家“中型互联网公司（或银行/制造业也行）”做一个内部智能助手，解决以下问题：

- 员工经常问：制度流程、报销规则、技术文档、项目进度……
- 现在靠 FAQ 文档 + 问人，效率很低。
- 目标：做一个 “**能看文档、能查系统、还能自动帮你办事**” 的 AI 助理。

这个项目要求你从 **产品视角** + **工程视角** 同时出发，最终交付一个可演示的系统初稿（含代码+设计文档）

二、最终交付物

1. **产品文档**
    - 《项目背景与用户画像》
    - 《PRD / 需求文档》
    - 核心流程图 & 原型图（可以用墨刀/Figma/手绘都行）
2. **技术实现**
    - 一套可运行的 Demo（本地或云端均可）
    - 完成：
        - RAG 检索问答
        - 至少 2 个 MCP 工具（如：查询数据库 / 调用 HTTP API / 读写文件）
        - 至少 1 个多步 Agent 流程（自动拆分任务 + 调用工具）
3. **架构与技术说明**
    - 系统整体架构图
    - MCP 工具协议说明（tools / server / 调用示意）

4. **评估与复盘**
    - 至少 5 组真实 Query 的效果截图/记录
    - 遇到的坑、优化点、下一步版本规划

三、分阶段任务设计

Phase 0：产品定义 & 场景选型（AI 产品设计）

作业 0：定义你这款 AI 助理的“人设”和边界

1. 选一个具体场景（任选其一，也可以自定）：
    - 场景 A：“**制度 & 流程智能问答 + 自动生成邮件/申请单**”
    - 场景 B：“**技术文档问答 + 自动生成接口调用示例**”
    - 场景 C：“**客户服务知识库问答 + 自动生成回复模版**”
2. 写一份 **1–2 页的《产品背景 & 用户画像说明》**：
    - 用户是谁？（比如：HR、研发、客服）
    - 他/她每天有哪些高频问题？（列出 5–10 条真实问法）
    - 现有解决方式是什么？痛点在哪里？
3. 写一份简要 **PRD**（1–2 页即可）：
    - 核心价值主张：这款 AI 助理到底帮用户节省了什么？
    - 核心功能列表（优先级：P0/P1）
    - 非功能需求：安全/合规、响应时间、准确率预期等
4. 画一张 **用户使用流程图**：
    - 从“用户发起问题/任务”到“拿到结果/完成操作”的全过程

> 要求：这一阶段**完全站在产品视角**，暂时不纠结技术细节，用业务语言把问题说清楚。

Phase 1：知识库 & RAG 层设计（RAG + LlamaIndex/LangChain）

作业 1：构建一个“可用”的企业知识库并完成检索问答

1. 数据准备
    - 收集一批“模拟企业文档”（可以是在线找的制度 PDF / 技术文档 / FAQ，自行整理）
    - 至少包括两种格式：PDF / Markdown / Word / 网页之一
2. 选框架 & 设计：
    - 要求：**LlamaIndex & LangChain 至少用到一个，最好两个都用**：
        - 方案示例：
            - 方案 A：用 **LlamaIndex** 做索引 + 检索，LangChain 做 Agent 编排
            - 方案 B：两种框架分别做一个 RAG 流水线，对比检索效果
    - 写清楚：
        - 使用了哪种 chunk 策略？（按页 / 按标题 / 按语义切分）
        - Embedding 模型、向量库选择（可以用开箱即用向量库）
3. 实现基础 RAG 问答：
    - 用户输入自然语言问题 → 系统检索相关文档片段 → 大模型生成回答
    - 要求：
        - 支持**中文问答**
        - 回答中给出**引用来源**（文档名 + 段落）
4. 对比 & 反思（如果用了两个框架）：
    - 在 5–10 个真实 Query 上比较：
        - 哪种方案召回更准？哪种更稳定？哪种更好集成进 Agent？

> 交付：一段可运行代码 + 效果截图 + 简单对比表。

Phase 2：接入 MCP 工具（MCP）

作业 2：设计并实现至少两个 MCP 工具

目标：让你的 AI 助理不仅能“答题”，还能调用工具。

1. 选 MCP 工具类型（任选其二，或自己设计）：
    - 工具 1：**HTTP API 工具**

        - 例如：调一个模拟的“流程状态查询接口”“天气接口”查询地图”报销系统接口”等
    - 工具 2：**数据库 / 文件系统 工具**
        - 例如：查询 SQLite 中的“员工表/客户表/订单表”
    - 工具 3：**任务创建工具**
        - 如：往一个 JSON 文件/数据库里写一条“工单/任务”记录，模拟创建 Jira/工单
2. MCP 设计文档：
    - 列出每个 Tool 的：
        - name、description（用业务语言）
        - 输入参数（schema）
        - 返回结构（schema）
    - 画一张 “MCP Server 结构图”（简单即可：Client ↔ MCP Server ↔ 外部资源）
3. 实现 MCP Server：
    - 使用官方推荐的一个语言/SDK（如 TypeScript / Python MCP SDK）
    - 确保你的 LLM 客户端可以通过 MCP 调用这些工具
4. 测试用例：
    - 设计 5 条自然语言指令，让模型自动决定是否调用工具
    - 例如：
        - “帮我查一下张三 3 月份的报销总金额，并生成一段邮件说明。”
        - “查询一下《费用报销制度》的报销上限，然后帮我写个解释给领导。”

> 交付：工具定义文档 + 代码 + 实际调用日志（截图/记录）。

Phase 3：Agent 设计与编排（Agent + LangChain / LlamaIndex）

作业 3：构建一个“多步骤工作流 Agent”

目标：让助理能“自己想步骤”，而不是单轮问答。

1. 设计一个典型任务流程（举例）：
    - 用场景 A（制度问答 + 自动邮件）举例：

    1. 理解用户意图：是“问规则”还是“要帮写东西”？
    2. 若需要规则 → 调 RAG 检索制度文档
    3. 若与某人/某部门相关 → 调 MCP DB 工具查数据
    4. 自动草拟邮件/申请单 → 输出给用户确认
2. 选择 Agent 实现方式：
    - 可以用 **LangChain 的 Agent / LangGraph / 自己写 loop** 任意一种
    - 要求：
        - Agent 能够在一次对话中**多次调用工具**
        - 工具调用的选择是由模型“reasoning”后决定的（不是写死的 if-else）
3. 实现一个“端到端流程”：
    - 用户一句话：
        - 例：“我想申请差旅报销，帮我确认下我是否符合报销条件，并帮我写一封发给 HR 的邮件。”
    - 系统自动：
        - 去 RAG 查规则 → 用 MCP 查个人数据 → 生成总结 → 合并结果输出

> 交付：Agent 代码 + 示例对话 + 工具调用轨迹说明。

Phase 4：产品体验优化 & 评估（AI 产品设计 + 技术）

作业 4：从“产品 +技术”双视角做一次复盘

1. 体验评估（至少 5 个用户任务）：
    - 为你的系统设计 5–10 个“真实任务模板”，比如：
        - “给新人解释差旅报销流程。”
        - “查询张三 3 月份加班工时报表并生成汇总说明。”
    - 手动跑一遍，记录：
        - 是否完成任务？
        - 交互轮数？

        - 有哪些误解或错误？
2. 建立简易指标：
    - 准确性：回答是否符合文档/业务规则（自评）
    - 完成率：任务是否完全闭环
    - 操作成本：平均需要几轮对话/几次工具调用
3. 找出 3–5 个关键问题：
    - 比如：
        - RAG 召回错文档优化方法？
        - Agent 步骤规划错误怎么办？
        - MCP 工具参数理解有偏差是为什么？
4. 输出一份《版本迭代计划》：
    - V1 → V1.1：你会怎么改？
    - 可以从：数据、提示词、工具设计、Agent 策略、UI 交互等方面给出改进点

> 交付：评估表 + 改进建议文档（1–2 页）。

四、项目结构建议

1. **项目题目**
    - 《企业智能知识助理 & 工作流 Agent 系统（MCP + RAG + Agent 综合项目）》
2. **项目目标（1–2 段话）**
3. **阶段任务 & 验收标准**
    - Phase 0：产品定义（交付：背景 + PRD + 流程图）
    - Phase 1：RAG 知识库（交付：代码 + 效果对比）
    - Phase 2：MCP 工具（交付：工具定义 + 日志）
    - Phase 3：Agent 流程（交付：Agent 代码 + 端到端示例）
    - Phase 4：评估与迭代（交付：评估表 + 版本计划）

4. **技术栈硬性要求**
    - 必须使用：
        - 至少 1 个框架应用 LlamaIndex / LangChain
        - 至少 1 个 MCP 工具
    - 语言自选：Python
5. **加分项**
    - 实现一个简单 Web UI（如 Streamlit / Gradio ）